Análise de Algoritmos e Notação Assintótica

Fernando Garcia Diniz Campos Ferreira

Algoritmos

- Algoritmos são conjuntos de instruções ou passos finitos que visam resolver um problema ou realizar uma tarefa específica.

- Características de um Algoritmo:
  - Entrada.
  - Saída.
  - Definição Precisa.
  - Efetividade.
  - Finitude.

Análise de algoritmos

- A análise de algoritmos busca avaliar a eficiência de algoritmos em relação ao seu **tempo de execução** e **uso de memória**.

- A análise de algoritmos é fundamental para:
  - Tomadas de decisões: escolha do algoritmo mais apropriado para determinado tipo de problema.
  - Otimização: encontrar pontos fracos em um algoritmo e melhorar seu desempenho.

Medida de tempo de execução de uma programa

- Conforme Knuth (1971), existem dois problemas distintos:
  - Análise de um Algoritmo Particular
    - Análise do custo de utilizar uma algoritmo para resolver um problema específico.
  - Análise de uma Classe de Algoritmos
    - Qual o algoritmo de menor custo para um problema em particular?

Medida de tempo de execução de uma programa

- O custo de utilização de um algoritmo pode ser medido de várias maneiras:
  - Mensurar tempo de execução diretamente, por meio da execução do programa.
    - As medidas obtidas são inadequadas, pois:
      - Os resultados são dependentes do compilador.
      - Os resultados dependem do hardware
      - Dependem da quantidade de memória.

Medida de tempo de execução de uma programa

- Utilizar um modelo matemático.
  - É baseado em um computador idealizado (não real).
  - O conjunto de operação a ser executadas é especificado, juntamente com o custo de cada uma.
  - Ignora-se custo de operações menos significativas.

- Por exemplo: na ordenação, considera-se apenas o **número de comparações**.

Função de Complexidade

- São utilizada para medir o custo de execução de um algoritmo.

- Função de Complexidade f, onde $f(n)$, é a medida de tempo necessário para se executar um algoritmo para um problema de tamanho n.

Função de Complexidade

- Função de Complexidade
  - de Tempo: (quando $f(n)$ *é uma medida da quantidade de tempo para executar um algoritmo*)
  - de Espaço: (quando $f(n)$ *é uma medida da quantidade de memória para executar um algoritmo*)

- A complexidade de tempo não representa na realidade o tempo diretamente, mas sim, o **número de vezes que determinada operação considerada relevante é executada**.

Função de Complexidade

- Determinando o menor custo possível para resolver problemas de uma classe, temos a medida da dificuldade inerente para resolver o problema.

- Quando o custo de um algoritmo é igual ao menor custo possível, o algoritmo é **ótimo** para a medida de custo considerada.

Função de Complexidade

- Exemplo: Algoritmo para encontrar o maior elemento de um vetor:

```
int Max(TipoVetor [] A)
{
    int i , Temp;
    Temp = A[0] ;
    for ( i = 1; i < N; i ++)
        if ( Temp < A[ i ] )
                Temp = A[ i ] ;
    return Temp;
}
```

  – $f(n)$ = número de comparações entre os elementos.

Função de Complexidade

- Exemplo: Algoritmo para encontrar o maior elemento de um vetor:

```
int Max(TipoVetor [] A)
{
    int i , Temp;
    Temp = A[0] ;
    for ( i = 1; i < N; i ++)
        if ( Temp < A[ i ] )
            Temp = A[ i ] ;
    return Temp;
}
```

  – $f(n)$ = número de comparações entre os elementos.

Função de Complexidade

- São necessárias $n - 1$ comparações para encontrar o maior elemento.

- Como $n - 1$ é o menor número possível de comparações, **então o algoritmo é ótimo**.

- $f(n) = n - 1$, para $n > 0$.

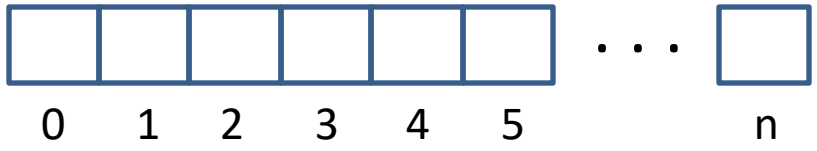
Função de Complexidade

- A medida de custo de execução de um algoritmo depende principalmente do tamanho da entrada dos dados.

- Portanto, há três cenários:
  - Melhor Caso
  - Pior Caso
  - Caso Médio (ou Esperado)

Função de Complexidade

- Exemplo: algoritmo de pesquisa sequencial em arquivo:
  - Dada uma chave qualquer, encontrar o registro que contenha aquela chave:



  - O algoritmo examina os registros, um a um, até que o registro procurado seja encontrado (ou não, caso não exista).

Função de Complexidade

- Seja f uma função de complexidade tal que $f(n)$ é o número de registros consultados no arquivo, tem-se:
  - Melhor caso: $f(n) = 1$
  - Pior caso: $f(n) = n$
  - Caso médio: $f(n) = \frac{(n+1)}{2}$

- Para o caso médio, se cada registro tiver a mesma probabilidade de ser acessado que todos os outros, temos:

$$f(n) = \frac{1}{n}\,(1 + 2 + 3 + \ldots + n) = \frac{1}{n}\left(\frac{n(n+1)}{2}\right) = \frac{n+1}{2}.$$

Comportamento Assintótico de Funções

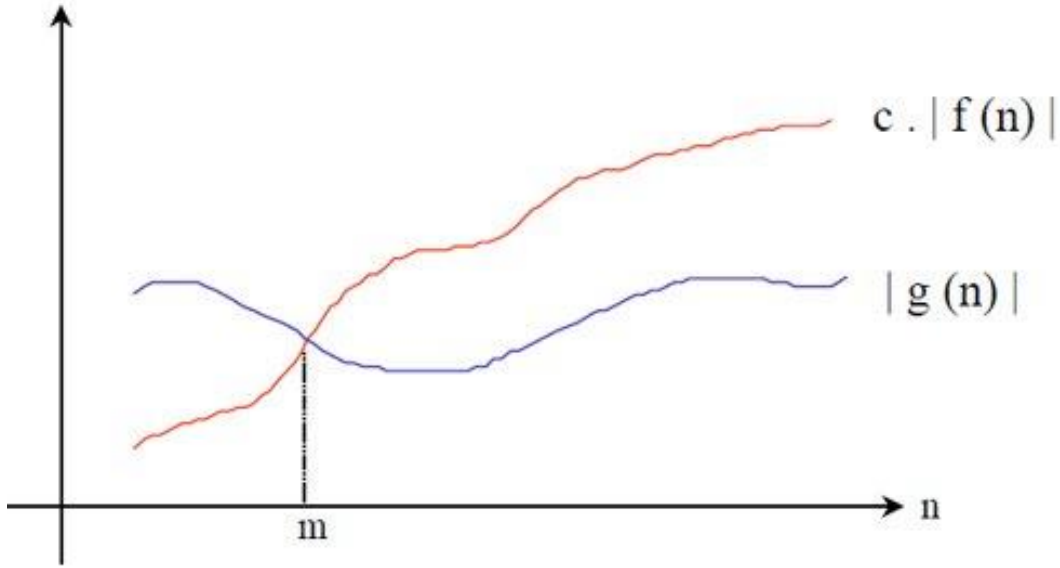
- O custo para se obter uma solução aumenta com o tamanho da entrada (n) do problema.

- O parâmetro n fornece uma medida da dificuldade para se resolver o problema.

- A análise de algoritmos é realizada para valores grandes de n.

Comportamento Assintótico de Funções

- Por definição: A função *f(n)* domina assintoticamente a função *g(n)* se existem duas constantes positivas *c* e *m* tais que, para $n \geq m$, tem-se: $|g(n)| \leq c \times |f(n)|$.
- Notação: $g(n) = O(f(n))$

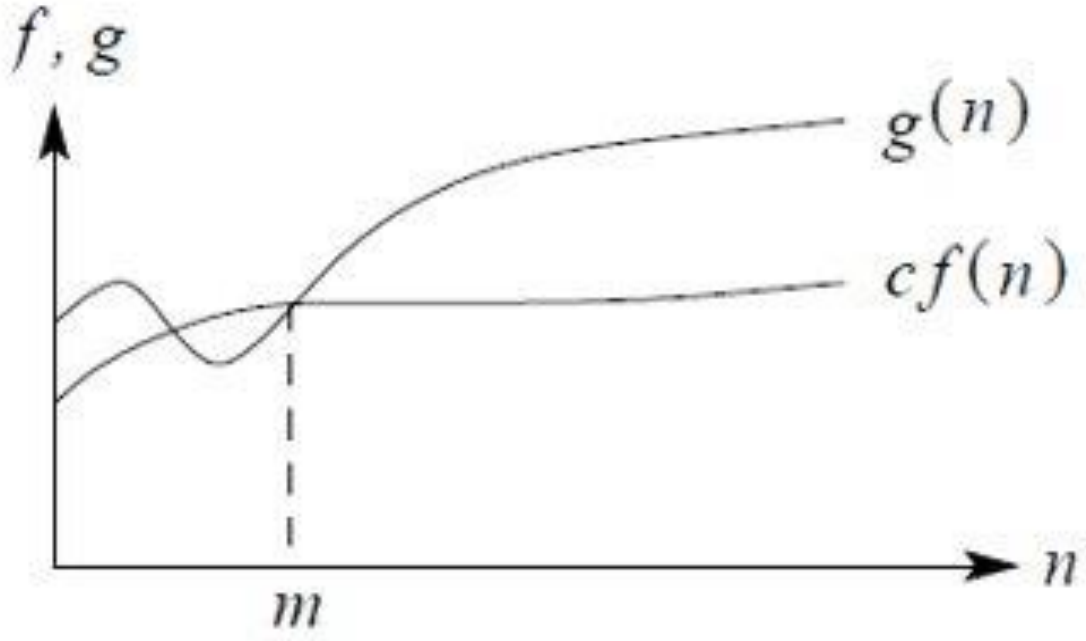


Operações com a Notação O

$$
\begin{aligned}
f(n) &= O(f(n)) \\
c \times O(f(n)) &= O(f(n)) \quad c = constante \\
O(f(n)) + O(f(n)) &= O(f(n)) \\
O(O(f(n)) &= O(f(n)) \\
O(f(n)) + O(g(n)) &= O(max(f(n), g(n))) \\
O(f(n))O(g(n)) &= O(f(n)g(n)) \\
f(n)O(g(n)) &= O(f(n)g(n))
\end{aligned}
$$

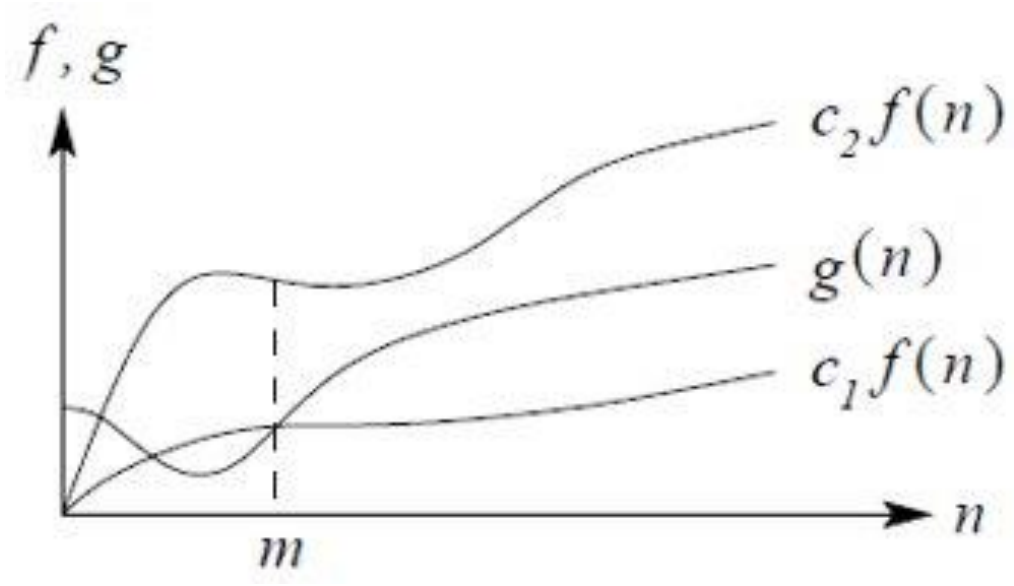
Comportamento Assintótico de Funções

- Uma função g(n) é Ω(f(n)) se existirem duas constantes c e m tais que g(n) ≥ cf(n), para todo n ≥ m.
- Limite inferior.



Comportamento Assintótico de Funções

- Limite Superior e Inferior (Θ).
- Uma função g(n) é Θ(f(n)) se existirem constantes c_1, c_2 e m tais que $0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)$, para todo $n \geq m$.



Complexidade assintótica

- Se f é uma **função de complexidade** para um algoritmo F, então O(f) é considerada a **complexidade assintótica** do algoritmo F.

- Os algoritmos que possuem a mesma **complexidade assintótica**, pertencem a mesma **classe de complexidade**.

Classes de complexidade

- $f(n) = O(1)$ – Constante
  - Uso do algoritmo independe de n.
  - As operações são realizadas um número fixo de vezes.

- $f(n) = O(\log n)$ - Logarítmica
  - Típico em algoritmos que quebram um problema em outros menores.
  - Busca binária em um conjunto ordenado.

Classes de complexidade

- $f(n) = O(n)$ – Linear
  - Em geral, um pequeno trabalho é realizado sobre cada elemento de entrada.
  - É a melhor situação possível para um algoritmo que tem de processar/produzir n elementos de entrada/saída.
  - Algoritmo de busca linear, encontrar o máximo de um conjunto.

- $f(n) = O(n \log n)$
  - Típico em algoritmos que quebram um problema em outros menores, resolvem cada um deles independentemente e combina as soluções depois.
  - Merge sort, heap sort.

Classes de complexidade

- $f(n) = O(n^2)$ – Quadrática
  - Ocorrem quando os itens são processados aos pares, muitas vezes em um loop dentro de outro.
  - Bubble sort.

- $f(n) = O(2^n)$ – exponencial
  - Podem não ser úteis sob o ponto de vista prático.
  - Envolvem soluções de problemas quando se usa força bruta.

Classes de complexidade

Função de Complexidade*	Tamanho do Problema (n)					
	10	20	30	40	50	60
n	0,00001 segundos	0,00002 segundos	0,00003 segundos	0,00004 segundos	0,00005 segundos	0,00006 segundos
n^2	0,0001 segundos	0,0004 segundos	0,0009 segundos	0,0016 segundos	0,0025 segundos	0,0036 segundos
n^3	0,001 segundos	0,008 segundos	0,027 segundos	0,064 segundos	0,125 segundos	0,216 segundos
n^5	0,1 segundos	3,2 segundos	24,3 segundos	1,7 minutos	5,2 minutos	13,0 minutos
2^n	0,001 segundos	1,0 segundos	17,9 minutos	12,7 dias	35,7 anos	366 séculos
3^n	0,059 segundos	58 minutos	6,5 anos	3855 séculos	2×10^8 séculos	$1,3 \times 10^{13}$ séculos

Garey e Johnson (1979)

Exemplos

- $g(n) = (n + 1)^2$
  - $g(n)$ é $O(n^2)$, quando $m = 1$ e $c = 4$.
  - Porque $(n + 1)^2 \leq 4n^2$ para $n \geq 1$.

- $g(n) = 3n^3 + 2n^2 + n$
  - $g(n)$ é $O(n^3)$, sendo que $3n^3 + 2n^2 + n \leq 6n^3$ para $n \geq 0$.

- $g(n) = 3n^3 + 2n^2$
  - $g(n)$ é $\Omega(n^3)$. Basta fazer $c = 1$, e então $3n^3 + 2n^2 \geq n^3$ para $n \geq 0$.

Exercício

- Identificar a complexidade assintótica do algoritmo:

```
void MaxMin1 (TipoVetor[] A, int Max, int Min)
{
    int i;
    Max = A[0];
    Min = A[0];

    for ( i = 1; i < N; i++)
    {
        if ( A[i] > Max )
                Max = A[i] ;
        if ( A[i] < Min )
                Min = A[i] ;
    }
}
```

Exercício

- Identificar a complexidade assintótica do algoritmo:

```
void MaxMin2 (TipoVetor[] A, int Max, int Min)
{
    int i;
    Max = A[0];
    Min = A[0];

    for ( i = 1; i < N; i++)
    {
        if ( A[i] > Max )
                Max = A[i] ;
        else if ( A[i] < Min )
                Min = A[i] ;
    }
}
```

- A segunda comparação só é necessária quando a anterior é falsa.

Exercício

- Comparação entre os três algoritmos:

Algoritmos	***f(n)***		
	Melhor Caso	Pior Caso	Caso Médio
MaxMin1	$2(n - 1)$	$2(n - 1)$	$2(n - 1)$
MaxMin2	$n - 1$	$2(n - 1)$	$3n/2 - 3/2$
MaxMin3	$3n/2 - 2$	$3n/2 - 2$	$3n/2 - 2$

Referências

- Knuth, D.E. (1971), An empirical study of FORTRAN programs. https://doi.org/10.1002/spe.4380010203.
- Cormen, T., Leiserson, C., Rivest, R., Stein, C. Algoritmos - Teoria e Prática (3a. Edição). Editora GENLTC, 2012.
- Drozdek, A. Estrutura de Dados e Algoritmos em C++ (2a. Edição). Cengage Learning, 2016.
- Ziviani, N. Projeto de algoritmos com implementaçao em java e c++. Cengage Learning, 2006.